

Explainability in AI-based behavioral malware detection systems

Antonio Galli, Valerio La Gatta, Vincenzo Moscato, Marco Postiglione, Giancarlo Sperli *

Department of Electrical Engineering and Information Technology (DIETI), University of Naples "Federico II", Via Claudio 21, Naples, Italy

ARTICLE INFO

Keywords:

Behavioral malware detection
EXplainable artificial intelligence
Data-driven cyber security
Long-short term memory

ABSTRACT

Nowadays, our security and privacy are strongly threatened by *malware* programs which aim to steal our confidential data and make our systems out of service, among other things. While traditional signature-based malware detection methods or statistical analysis have proven to be ineffective and time-consuming, recently data-driven *Artificial Intelligence* (AI) techniques, i.e. *Machine Learning* (ML) and *Deep Learning* (DL) approaches, have been successfully applied leveraging the behavior of malware in terms of API calls, and achieving promising performances. However, their black-box behavior leads to a lack of explainability thus preventing their application in real world scenarios. In light of this, *eXplainable Artificial Intelligence* (XAI) methodologies and tools can be effectively embedded within an AI-based malware detection process in order to make more understandable the produced results. In this paper, we propose a XAI framework for behavioral malware detection problems and evaluate the usefulness of four XAI methods (SHAP, LIME, LRP and Attention mechanism) on three datasets with different size, sequence length and number of classes, by which we could evaluate the strengths and weaknesses – from effectiveness and efficiency point of views – of recurrent deep architectures (i.e. Long-Short Term Memory (LSTM) and Gated Recurrent Unit (GRU) models), and their applicability in the modern *Cyber Security* (CS) scenarios.

1. Introduction

In the last decade, the number and variety of cyber attacks to Information Systems of organizations or single individuals has steadily grown, mainly due to the diffusion of Internet of Things (IoT) devices (Fei et al. (2023)) and Cloud Computing (Karn et al. (2021)).

Cyber criminal attacks may have countless purposes, including the insertion of advertising and purchase offers in websites, the denial of service of computing systems or the stealing of confidential data for financial gains. Generally, the attackers leverage malicious software (e.g. trojan, backdoor, spyware, worm), also known as *malware*, to cause serious threats for the vulnerability of systems (Yan et al. (2023)).

The detection of malware is an extremely challenging research task because as Cyber Security (CS) researchers develop new techniques, malware authors improve their ability to evade detection (Galloro et al. (2022); Zhong et al. (2023)). From this perspective, the Cyber Threat Intelligence (CTI) process aims to analyze and monitor the huge volumes of cyber security information (e.g. execution logs, traffic logs) to enhance the security of organizations' systems (Sun et al. (2023)).

Traditional antivirus software programs cannot constitute an effective and timely solution to malware detection, since they are designed to detect viruses by searching their signature within a database which is updated from time to time (*signature-based malware detection* (Bazrafshan et al. (2013))). Such approaches are easily escaped with advanced evasion techniques, such as *code obfuscation* (Cheng et al. (2021)).

From a more structured perspective, malware signatures can be analyzed in a *static* or *dynamic* fashion: while Static Malware Detection (SMD) approaches focus on the software's byte-code, Behavioral Malware Detection (BMD) analyses the behavior of the software at runtime, i.e. the sequence of Application Programming Interface (API) calls which the software makes to the operating system (Alazab et al. (2010)) (refer to Fig. 1 for an example). Specifically, API is a set of protocols and functions to exchange data between different devices and applications (as shown in the example of Fig. 1), while a system call provides an interface between the underlying operating system and the program. Hereafter, we refer to API-calls and system calls as synonyms. The complex and subtle behavior of modern malwares makes SMD vulnerable to several attacks (e.g. dynamic code linking and encryption) (Gaber et al. (2023)) and explains the increasing interest in designing (semi-)au-

* Corresponding author.

E-mail addresses: antonio.galli@unina.it (A. Galli), valerio.lagatta@unina.it (V. La Gatta), vincenzo.moscato@unina.it (V. Moscato), marco.postiglione@unina.it (M. Postiglione), giancarlo.sperli@unina.it (G. Sperli).

<https://doi.org/10.1016/j.cose.2024.103842>

Received 13 October 2023; Received in revised form 22 March 2024; Accepted 2 April 2024

Available online 5 April 2024

0167-4048/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

```

InternetOpenA InternetOpenUrlA InternetCloseHandle
OpenSCManagerA CreateServiceA StartServiceA
LdrGetDllHandle LdrGetProcedureAddress FindResourceA
LoadResource SizeofResource NtOpenFile NtClose
NtCreateFile NtWriteFile NtClose CreateProcessInternalW
LdrGetDllHandle NtTerminateProcess NtClose
NtUnmapViewOfSection NtClose RegCloseKey NtClose
NtUnmapViewOfSection NtClose RegCloseKey NtClose
LdrUnloadDll NtOpenKey NtQueryValueKey NtClose
LdrUnloadDll NtClose NtTerminateProcess

```

Fig. 1. Scores assigned to API calls of a *ransomware* sequence. Red and blue tokens have a positive and negative influence to the prediction, respectively. The color intensity indicates the level of importance of the token. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

tomatic system to perform BMD. In this context, statistical approaches (e.g. n-grams (Santos et al. (2009)), Hidden Markov Models (Imran et al. (2015))) are often ineffective and time-consuming (Damodaran et al. (2017)) and, consequently, Machine Learning (ML) and Deep Learning (DL) are currently being adopted to advance the state-of-the-art detection systems (Catak et al. (2020); Gopinath and Sethuraman (2023); Gaber et al. (2023)).

Despite their outstanding performance, these approaches usually result in complex models that neither their designers are able to understand. While maintaining high levels of learning performance, it would be good — and sometimes mandatory — to guarantee the explainability of models, i.e. their ability to be understood and appropriately trusted by humans (Ali et al. (2023)).

Consequently, we are assisting to a more and more pervasive diffusion of *eXplainable Artificial Intelligence* (XAI) methodologies and tools, even in the cyber-security domain (Zhang et al. (2022)). When considering malware analysis, the explainability requirements have been surveyed in the context of Android malware defense models (Liu et al. (2022a); Fan et al. (2021); Wu et al. (2021); Liu et al. (2022b)) and some explainable approaches have been proposed for SMD (Fan et al. (2021); Liu et al. (2022b)).

However, the explainability problem for BMD systems is still under-explored, especially when the detection model employs modern deep sequential neural networks. In light of this, our analyses consider the sequential behavior of a malware, which is different from the traditional static approach based on feature extraction as demonstrated in (Fan et al. (2021); Wu et al. (2021); Liu et al. (2022b)). Additionally, while previous work (Fan et al. (2021); Wu et al. (2021); Liu et al. (2022b)) only focused on classifying Android applications as malware or not, our approach provides a more comprehensive view by investigating a broader range of systems. Concretely, XAI could support a variety of analyses of malware's behavior, e.g. by ranking its API calls according to how important they were for the model to make its prediction. Eventually, the explanation might help the analyst to understand the pivotal malware operations along with the affected resources. A simple example is shown in Fig. 1: the explanation suggests that the most relevant API considered by the model to classify the software behavior as a *ransomware* is *InternetOpenUrlA*.

In this paper, we present a XAI framework for BMD, aiming to evaluate the effectiveness, strengths, and weaknesses of four distinct XAI methods. We compare explanations provided by Local Interpretable Model-agnostic Explanations (LIME), SHapley Additive exPlanation (SHAP), Layer-wise Relevance Propagation (LRP), and attention mechanisms when explaining predictions from LSTM and GRU models. This model processes API call sequences as input and outputs malware class predictions. We quantitatively assess the similarity of explanations provided by different techniques and validate the effectiveness of explanations by evaluating how predictions change when influential features are retained or non-influential features are removed. Our experiments span three publicly-available datasets with varying sizes, sequence lengths, and class numbers, revealing the efficacy of each

method under diverse experimental perspectives despite disagreements between explanations.

The paper is organized as follows: Section 2 discusses related work on deep learning techniques applied to malware detection and some preliminary works considering XAI in this domain. Section 3 contains a comprehensive description of LSTMs and the explanation techniques evaluated throughout this work, while a detailed description of our framework is provided in section 4. Our experiments and results are detailed in Section 5. Finally, Section 6 reports conclusions and possible future work.

2. Related work

The data-driven nature of the CTI process has determined an increasing interest in applying machine learning and deep learning approaches to support its activities (Mills et al. (2022); Sun et al. (2023)). In particular, the detection of malware activities is becoming a challenging task due to the explosive growth in the number of connected devices and the evolution of attack strategies (Fei et al. (2023)).

In the last years, an increased number of works is focusing on BMD due to the robustness of such approaches to several types of attacks (e.g. code obfuscation, dynamic code linking and encryption) (Gaber et al. (2023)). Common deep learning methods dealing with the dynamic nature of system calls are based on Convolutional Neural Network (CNN), Recurrent Neural Networks (RNNs), LSTMs, and Graph Convolutional Networks (GCNs) (Gopinath and Sethuraman (2023)). Table 1 summarizes the state-of-the-art BMD approaches categorized by classification model.

Despite having proven promising detection performance, the aforementioned methods could not be used in real scenarios due to the lack of explainability caused by their complexity; as a matter of fact, the complexity of AI models and their behaviors is often obscure to end users having to make decisions based on AI-based predictions. Consequently, researches have now converged into the XAI field, whose aim is to produce intelligent agents able to provide explanations for their decisions (Saeed and Omlin (2023); Ali et al. (2023)). In particular, the term explainability refers to the ability to explain or to visualize an AI model's decision in understandable terms to humans (e.g. Rong et al. (2023)).

Various works investigate the explainability issue in malware detection: (Melis et al. (2018)) propose a gradient-based approach to identify the most influential features in the detection process; (Han et al. (2019)) investigate the correlation between static and dynamic features that affect prediction performance about different malware families. Furthermore, (Warnecke et al. (2020)) evaluate six explanation methods aiming to unveil the main features in deep learning models for malware detection.

However, to the best of our knowledge, the explainability problem has not been investigated yet in the BMD domain. In fact, our analyses on explanation methods consider the sequential behavior of a malware, which is different from the classical static approach based on feature extraction as demonstrated in (Fan et al. (2021); Wu et al. (2021); Liu et al. (2022b)). Additionally, while current literature (Fan et al. (2021); Wu et al. (2021); Liu et al. (2022b)) mainly focuses on Android malware, our approach provides a more comprehensive view. In this paper, we explore the application of a comprehensive set of explanation techniques on top of LSTM-based deep learning models because of their wide use and high performance on several datasets and contexts (Jahromi et al. (2020)), also confirmed by our experimental analysis. In particular, we compared the explanations provided by four different state-of-the-art XAI methods, relying on feature importance, with three different datasets varying in size, sequence length and number of classes.

Table 1
State-of-the-art approaches for behavioral malware classification.

Model	Paper	Approach
RNN	Jha et al. (2020)	The analysis relies on three feature vectors (one-hot encoding, random and Word2Vec), underlying the relevance of input size parameter for the model.
	Marques et al. (2021)	It provides an analysis about the application of RNN models for malware detection.
CNN	Jeon and Moon (2020)	It investigates opcode sequences by combining static and dynamic low-level features.
	Yadav et al. (2022)	A CNN-based model is proposed to detect Android malware using image-based malware representations of the Android DEX file.
GCN	Pei et al. (2020)	<i>AMalNet</i> relies on GCNs for learning semantic and sequential patterns.
	de Oliveira and Sassi (2023)	It performs a dynamic malware analysis using a directed cyclic graph of the API call sequences, leveraging both temporal and spatial information.
LSTM	Jahromi et al. (2020)	It aims to deal with malware detection problem by combining global and short dependencies of input and using a regularization method for speed up method convergence.
	Catak et al. (2020)	LSTM model is used for classifying malware in 8 families based on sequential data extracted from API calls made on Windows operating systems.
	Li et al. (2022a)	A BI-LSTM model has been designed to extract API relationship information generated through an embedding and convolutional layers.
Ensemble	Chakraborty et al. (2020)	It combines clustering and classification tasks for identifying malware families.
	Bhat et al. (2023)	Ensemble of machine learning models has been designed to classify Android malware based on system calls, binders and Android objects.
	Demirkiran et al. (2022)	Ensemble of deep learning models has been proposed for multiclass malware detection based on system calls.

3. Theoretical background

In this section we introduce some basic notions, which our work relies on. In particular, we firstly describe LSTMs — the deep learning technique at the core of our malware classifiers — and we successively focus on the explanation techniques analyzed throughout this paper.

3.1. Recurrent neural networks

As mentioned in the previous sections, behavioral malware detection deals with API calls resulting from the execution of malware and goodware instances in sand-boxed environments. The sequential nature of these data makes RNNs and LSTMs particularly suitable to perform the detection task. The general idea behind these models is to introduce a dependence mechanism between consecutive time-steps, e.g. a malicious behavior is never determined by an API call in isolation, but by the sequence in which it takes place.

Specifically, LSTMs (Hochreiter and Schmidhuber (1997)) are an evolution of RNNs designed to learn long-term dependencies. Specifically, each time step t , i.e. each API call in the sequence, is processed through a memory cell C_t which store the information to remember in the successive time-steps. C_t 's values are controlled with three gates: forget f_t , input i_t and output o_t gates. In turn, f_t and i_t control C_t to decide what information should actually be thrown away and added, respectively. Finally, o_t filters which part of C_t should be passed to the next time-step. For instance, consider the API (sub-)sequence *NtTerminateProcess* → *NtOpenFile* → *NtWriteFile* and assume the current time-step is the second API call ($t = 2$). Under this settings, f_2 could decide to forget the previous one if it terminates a not-related process, i_2 might choose to store the target file information and o_2 could decide to output only the file privileges which is important to understand the legitimacy of the successive writing operation.

Based on the above-mentioned concepts, some LSTM variants, such as Bidirectional-LSTM (BiLSTM), Gated Recurrent Unit (GRU), have been proposed (Li et al. (2022b)). In particular, BiLSTM is a sequence processing model that consists of two LSTMs one taking the input in a forward direction and the other in a backwards direction, while GRU has only two gates that are reset and update making it less complex than LSTM in terms of parameters.

3.2. Explanation methods

Since our work is the first one focusing on XAI for behavioral malware detection, we decide to investigate strengths and weaknesses of the most famous state-of-the-art XAI techniques. Since there is no agreement on the best techniques, nor a standard for the explanation generation, we explore the whole XAI spectrum to assess whether in behavioral malware detection domain some techniques could be preferred to the others.

Concretely, we analyze explanations provided by LIME (Ribeiro et al. (2016)), SHAP (Lundberg and Lee (2017)), LRP (Bach et al. (2015)) and attention mechanism (Bahdanau et al. (2015)) on LSTM-based classification models. The first two methods are post-hoc, i.e. they are applied after the model is built and trained, and model-agnostic, i.e. they look at f as it were a black-box, considering only its inputs and outputs. Practically, they can be useful when explaining a third-party model (such as those provided on Microsoft and Amazon clouds), only accessible on-demand. On the other hand, LRP, despite being a post-hoc technique, is a white-box technique since it requires the access to f 's inner structure. Concretely, it is suitable in debugging scenarios where the domain expert could assess what the model is learning. Finally, the attention mechanism requires changing f 's structure and thus re-training the whole system. An overview of the techniques is shown in Fig. 2.

The XAI methods studied in this work represent a diverse range of techniques. LIME focuses on local model explainability through perturb-

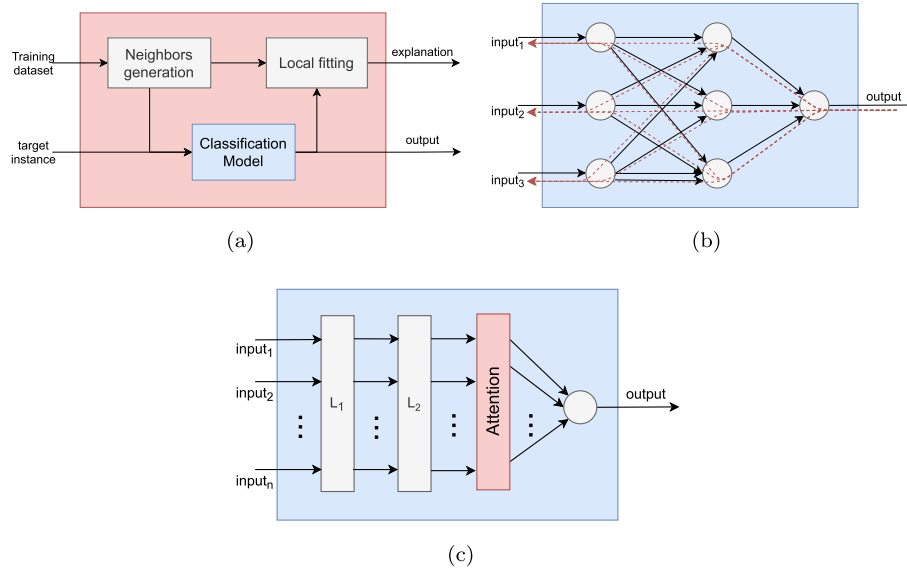


Fig. 2. Overview of the analyzed explanation techniques. Cyan boxes represent the scope of the model while red ones represent the scope of the explainer. LIME and SHAP (a) look at the model if it were a black-box, LRP (b) has access to the structure of the model while the attention mechanism (c) requires a new layer to be added to such structure.

ing input data, SHAP provides a game-theoretic approach to allocate contributions to each feature, LRP propagates relevance through neural network layers, and Attention mechanisms highlight input features attended to by the model. This diversity allows for a comprehensive exploration of different explainability approaches. Furthermore, the selected methods (LIME, SHAP, LRP, Attention) are particularly relevant when dealing with recurrent neural networks that are commonly used for behavioral malware detection. LIME and SHAP offer model-agnostic explainability, making them applicable to a wide range of models, including neural networks. LRP specifically addresses neural network layer-wise relevance, and Attention mechanisms are commonly used in neural networks for sequence data. LIME and SHAP have gained widespread adoption in the research community due to their versatility and effectiveness in explaining complex models. LRP is known for its application in neural networks, and Attention mechanisms have become standard in natural language processing and computer vision tasks. Choosing well-established methods enhances comparability with existing literature. Finally, all the experimented methods have readily available implementations and are user-friendly, making them suitable for application in real scenarios.

It is worth to note that all the analyzed techniques provide explanations in the form of feature-importance, i.e. they assign a score to each API call in the sequence, indicating how it contributes to f 's prediction (see the *ransomware* example in Fig. 1). Not only do feature-importance explanations are easily interpretable by end-users, but also they provide an effective summary, extracting just the small and typically not consecutive sub-sequences which determine the malware capability to harm the system. In the remainder of this section, we provide some details of the above-mentioned explanation methods.

3.2.1. Local interpretable model-agnostic explanations

LIME (Ribeiro et al. (2016)) creates a simplified and locally-interpretable surrogate model which approximates the complex behavior of the black-box model. It is based on the assumption that no matter how the decision function is complex, it can be approximated by a linear model in the neighborhood of the instance being the explanation target.

Formally, given an instance S , i.e. the API call sequence, its neighborhood π_S , generated by perturbing S elements, and the black-box model f , LIME finds an explainable model g (e.g. linear model) by minimizing the following objective function:

$$\xi(S) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_S) + \Omega(g), \quad (1)$$

$\mathcal{L}(f, g, \pi_S)$ being the unfaithfulness of g with respect to f and $\Omega(g)$ being g 's complexity. It is worth to note that, being a model-agnostic technique, LIME does not depend on f (e.g. LSTM, RNN, CNN) and it can be seamlessly applied to our LSTM-based model.

LIME's Achilles heel is the random generation of neighborhoods, which causes explanations of two very close instances to vary greatly (Alvarez-Melis and Jaakkola (2018)) and does not consider density of black-box outcomes in the neighborhood instances (Guidotti et al. (2018)). Despite this, LIME's explanations have been applied in various domains including text and/or sequence classification (Ribeiro et al. (2016); Kiefer (2022)) and cryptomining detection Karn et al. (2021), even considering LSTM models. Interestingly, (Liu et al. (2022b)) shows the promising performance of LIME explanations in the context of SMD.

3.2.2. Shapley additive exPlanations

SHAP (Lundberg and Lee (2017)) approximates the computation of Shapley values, a statistical method from coalition game theory (Shapley et al. (1988)), applied in different application domains (i.e., text classification (Kokalj et al. (2021)) and cryptomining detection (Karn et al. (2021))). At high level, features, i.e. the API calls in a sequence, are considered as "players" which cooperate in coalitions to obtain a profit and the prediction of the black-box model f provides the "payout" which has to be distributed among players. Shapley values are the marginal contribution of feature values across all possible coalitions. In other words, the i -th Shapley value represents the contribution of the i -th API call in the sequence for the prediction.

Specifically, three methods are proposed: (i) *kernelSHAP* is the model-agnostic version of the algorithm working only with inputs and outputs of f , (ii) *treeSHAP* computes the Shapley values assuming f is a tree-based classification model, (iii) *deepSHAP* optimizes the computation for neural network architectures.

3.2.3. Layer-wise relevance propagation

Initially designed to explain feed-forward neural networks (Bach et al. (2015)), LRP has been successively extended to CNN for image classification (Oksuz et al. (2022)) and LSTM for text classification (Arras et al. (2017)) and next activity prediction (Aversano et al. (2023)).

Its theoretical foundation is the "relevance conservation principle", according to which the total relevance sum of hidden neurons is the

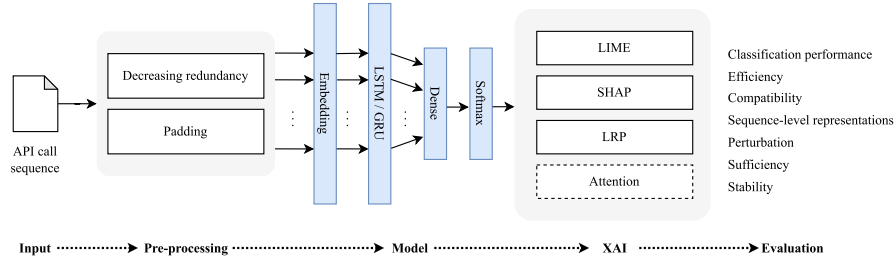


Fig. 3. Methodological workflow. The pre-processing step aims to standardize the data format. The model classifies the input sequence as malware/goodware, and the explainer generates the explanation. The models are then evaluated in terms of classification performance, efficiency and explanations quality.

same within each layer l of the network. As a result, given the (probability) output of the black-box model $f(S)$, S being the API calls sequence, the basic idea is to back-propagate the output such that $\sum_j R_j^{(l)} = f(S)$ for each layer l , $R_j^{(l)}$ being the relevance of the j -th neuron in the l -th layer. Eventually, $R_j^{(1)}$ will be the importance of j -th API call in the sequence, thus representing its contribution to the prediction.

LRP has been successively extended to deal with recurrent neural network architectures such as LSTMs (Arras et al. (2017)) by defining the redistribution rules dealing with the two types of neural connections involved by such networks: (1) many-to-one *weighted linear correlations* and (2) two-to-one *multiplicative interactions*, which are described as follows.

Weighted Correlations. The relevance R_i at the neuron z_i is computed by summing up all the messages $R_{i \leftarrow j}$ coming from upper-layer neurons $z_j = \sum_i z_i \cdot w_{ij} + b_j$.

Multiplicative Interactions. In multiplicative interactions, the value of the upper-layer neuron is computed as $z_j = z_g \cdot z_s$, where z_g and z_s are lower-layer neurons. When such kind of interaction appears in LSTMs, one of the two lower-layer neurons z_g is always a *gate*, whose values are in the range $[0, 1]$. The relevance is redistributed by setting $R_g = 0$ and $R_s = R_j$ because the aim of the gate neuron is to decide how much of the information contained in z_s has to be kept to make the decision in the forward pass, thus controlling the relevance which will be assigned to z_j by upper-layer neurons.

Differently from LIME and SHAP, LRP requires the access to the structure of the neural network, since it has to perform a backward pass through the network to compute the relevance scores.

3.2.4. Attention layer

Initially designed in the context of Machine Translation Bahdanau et al. (2015), the *attention mechanism* has been applied in many different domains, from document classification Yang et al. (2016) to scene segmentation Chen et al. (2023) to medical diagnosis Barata et al. (2021). In the context of text classification, its aim is to select the most relevant parts of context information to facilitate learning, assigning a score to each token of the sequence. Formally, we adopt the soft attention mechanism proposed in Yang et al. (2016).

It is worth to note that more powerful attention mechanisms, i.e. self-attention and multi-head attention Vaswani et al. (2017), have been recently proposed in NLP literature to learn meaningful representations even in unsupervised settings. We will compare these approaches in the experimental section.

However, according to the XAI taxonomy, the attention mechanism is not a post-hoc method, i.e. it requires the model to be built with an attention layer and hence trained on available training data.

Concretely, the attention scores reflect the relevance of each token for the prediction, thus being comparable with scores provided by the others feature importance-based techniques. However, it is worth to note that the contextual representation of the LSTM does not guarantee similar context vectors for similar sequences Du et al. (2019), thus representing a bias with respect to the other XAI post-hoc methods.

4. Method

4.1. Problem formulation

As mentioned above, behavioral malware detection analyzes the sequence of system calls (i.e. APIs) a malware/goodware invokes to access to the operating system's services. Formally, let $\mathcal{V} = \{API_i\}_{i=1, \dots, V}$ be the set of API names the operating system offers, then the malware/goodware can be described through the sequence $S_i = \{s_1, s_2, \dots, s_{m_i}\}$, $s_i \in \mathcal{V}$, m_i being S_i length. It is worth to note that the same API can be invoked a lot of times (even consequently) in the same sequence and different sequences might have different lengths.

Given the sequence S_i , the classification task aims to learn a model $f : S_i \rightarrow \mathcal{Y}$, $\mathcal{Y} \in \mathbb{R}^k$, k being the number of classes. This formulation, besides describing the classical binary setting of "malware vs goodware" problem, can be adopted for malware family (e.g. Trojan, Worm, Ransomware, etc.) classification as well.

Furthermore, we consider the problem of generating local feature-importance explanations for each sequence. In other words, the explanation task aims to score each API in the sequence according to how much it has contributed to the model's prediction. Thus, the explainer can be formalized as follows:

$$\mathcal{E} : (f, S_i) \rightarrow E_i,$$

$E_i \in \mathbb{R}^{m_i}$ being the vector such that $e_j \in \mathbb{R}$ represents the score of the j -th API in the sequence; the higher the score, the higher that API contributed to the model result. It is worth to note that the explainer module is not necessary different from the classification model itself: for instance, if f is transparent, the explanation step boils down to post-processing operations of the model results. Furthermore, negative scores imply that the corresponding system calls may have led the model towards another prediction w.r.t. the actual one.

4.2. Our framework

Our methodology builds upon a pipeline composed by three steps: the *sequence pre-processing* module aims to standardize the data format, the *model* is a classification learner which exploits the sequence structure of input data to perform the classification and the *explainer* generates the explanation supporting the model's prediction. Our methodological workflow is summarized in Fig. 3.

4.2.1. Sequence pre-processing

The original sequences collected from the sandbox execution suffer from different problems concerning indefinite sequence length, overlapping subsequences and blurred boundary of sequence from malware and benign software. We propose to perform the following operations:

- Decrease redundancy in the sequences by keeping only the first API call in a sub-sequence of identical calls in order to capture malware behavioral differences more clearly in sequence order. For instance, a malware attack could hide its core malicious sub-

sequence operations by interleaving repeatedly useless API calls (e.g. *SizeOfResource*, *FindResource*) with the threatening ones.

- Fix the sequence length by padding (head-truncating) the ones shorter (longer) than a threshold m . This operation is needed because sequential deep learning models (e.g. GRUs, LSTMs) take inputs of the same length and dimension during the training process (Dwarampudi and Reddy (2019)).

4.2.2. Model architecture

We consider classification models which should be able to process sequence data so as to extract knowledge from the API calls performed by malware/goodware. Specifically, we propose to use an LSTM- and GRU-based architectures, as depicted in Fig. 3. Given the fixed-length sequence $S \in R^{1 \times m}$, the embedding layer maps each input API s_i to a numeric vector $x_i \in R^d$, d being the embedding dimensionality. The LSTM/GRU layer transforms the (embedded) input sequence $X_i \in R^{m \times d}$ to a vector $X'_i \in R^{m \times d'}$, d' being the number of LSTM/GRU units. Finally, the dense and softmax layers produce the vector $y \in R^k$, whose j -th element represents the probability that the input sequence belongs to the j -th class.

4.2.3. Explainer

The explainer module works with the model and the instance to explain (i.e. target instance) in order to extract the explanation E as defined in the previous section. Depending on the explainer typology, there would be some considerations to be made: (1) model-specific techniques (Attention mechanism and LRP) force to use the classification model they are designed for, (2) white-box approaches (LRP) require the access to the internal states of the model, (3) model-agnostic methods (LIME, SHAP) do not rely on the classification model but may still require the dataset used for the training procedure. Consequently, different techniques may lead to dissimilar (even inconsistent) explanations since they make different assumptions and use diverse artifacts of the training stage.

5. Experimental evaluation

This section discusses the obtained results according to the experimental protocol, aiming to evaluate the proposed framework in terms of running time and efficacy with both quantitative and qualitative measures. In addition, we released our code in a github repository¹ to guarantee its replicability.

5.1. Experimental protocol

We evaluated both the efficiency and the effectiveness of the explanation methods described in Section 3.

Since the growing interest in applying of XAI methodologies for different application domains (e.g., propaganda detection (Cavaliere et al. (2023))), we focused our analysis from different points of view.

In particular, on the one hand, we compared the *run time* required to provide explanations (Section 5.3), on the other hand, the effectiveness of the different techniques has been assessed on both *qualitative* and *quantitative* points of view:

- we checked the extent to which explanations provided by different explainers are similar between each other by means of a *compatibility* experiment (Section 5.4.1);
- the *Sentence-Level Representation* approach proposed by (Arras et al. (2019)) has been applied to investigate whether it is possible to retrieve structured API call sequence representations through the scores provided by the analyzed techniques (Section 5.4.2);

- a *perturbation* experiment, inspired by (Arras et al. (2019)), has been performed to evaluate how predictions of the model change when deleting the mostly-influential tokens or keeping only them (Section 5.4.3);
- we compared XAI techniques based on their ability to provide deterministic explanations through a *stability* experiment (Section 5.4.4);
- a case study has been discussed to provide more insight about how XAI techniques can support the understanding of malware behavior (Section 5.5).

Our experiments have been performed on Google Colab² equipped with one single core hyper threaded Xeon Processor @2.2Ghz, 12 GB of RAM and a Tesla T4 GPU.

It is important to note that while the experiments on LIME, LRP and Attention have been executed with the Tensorflow version 2.3.0, we were obliged to a downgrade (to version 1.15.2) to run SHAP experiments due to compatibility issues.

5.1.1. Datasets

We ran our experiments on three datasets with different size, sequence length and number of classes. In particular, we have selected Mal-API-2019³ (Catak et al. (2020)), API Call Sequences⁴ (de Oliveira and Sassi (2023)) and Alibaba Cloud Malware⁵ (Alibaba Cloud (2018)) datasets, whose characteristics are summarized as follows.

Mal-API-2019 is a multiclass problem characterized by eight main malware families: Trojan, Backdoor, Downloader, Worms, Spyware Adware, Dropper, Virus. The dataset has been generated by searching the malwares' MD5 hash values on Github and, successively, using Virus-Total API in order to obtain the families of these malicious software.

Eventually, the API set cardinality $|\mathcal{V}|$ is 342, distributed among 7,107 sequences classified in the above-mentioned families.

API Call Sequences dataset contains 42,797 malware API call sequences and 1,079 goodware API call sequences, each one composed by the first 100 non-repeated consecutive API calls associated with the parent process, extracted from the 'calls' elements of Cuckoo Sandbox reports.⁶ In this case, the set of API $\mathcal{V}$ is formed by 307 unique API calls.

Alibaba Cloud Malware is a multiclass problem. It consists in API instruction sequences from the windows executable program files in the sandbox simulation. More in details, $|\mathcal{V}| = 298$ and the dataset contains a total of 600 millions of record, which are grouped into the following classes: Normal, Ransom, miner, DDoS, Worm, Virus, Backdoor and Trojan.

Table 2 reports sequence length distributions in terms of synthetic measures (mean, median, min and max) of each dataset before and after the removal of redundancies.

5.1.2. Models & explainers

The baseline classification model has been implemented in Tensorflow 2.0⁷ using the corresponding Keras layers. Table 3 reports the hyper-parameters configurations selected for each dataset. As introduced in Section 4.1, the trained predictors represent the complex and non-explainable models f which are, then, analyzed by the explainers e , whose configurations will be described in the following.

LIME⁸ has been configured so as to retrieve a score for each element of the sequence to explain (i.e. NUM_FEATURES = Sequence length). Be-

¹ To encourage reproducibility of our results, all code and datasets is publicly available on Github (<https://github.com/Anthony94/behavioralMalwareDetection>).

² <https://colab.research.google.com/>.

³ https://github.com/ocatak/malware_api_class.

⁴ <https://ieee-dataport.org/open-access/malware-analysis-datasets-api-call-sequences>.

⁵ <https://tianchi.aliyun.com/competition/entrance/231694/introduction>.

⁶ <https://cuckoosandbox.org>.

⁷ tensorflow2015-whitepaper.

⁸ <https://github.com/marcotcr/lime>.

Table 2

Sequence length distributions before and after the removal of redundancies.

Dataset	Mean	Median	Min	Max
Mal-API-2019	19647	633	10	1764421
→	19647	633	10	1764421
API Call Sequences	100	100	100	100
→	100	100	100	100
Alibaba	6716	1796	3	888206
→	3056	669	3	511789

Table 3

Hyper-parameters choices.

Hyper-parameter	Mal-API-2019	API Call Sequences	Alibaba
Sequence length	128	100	669
Embedding size	32	8	64
LSTM units	16	128	128

sides, we follow the author's suggestion to generate 5000 neighbors to capture the locality of the explanation. Since the developed model is a deep neural network, we adopt DeepSHAP⁹ to perform SHAP experiments.

LRP has been applied to our LSTM-based model by adapting the implementation¹⁰ provided by (Arras et al. (2017)). We set the hyper-parameters to $\epsilon = 0.001$, $\delta = 0.0$ based on the suggestions reported by the authors.

Finally, two versions of the attention layer have been compared, namely the soft attention mechanism presented in (Yang et al. (2016)) and the more recent multi-head attention proposed in Vaswani et al. (2017). The model architecture has been retrained positioning the attention layer right after the LSTM one, considering two heads for the multi-head attention layer.

5.2. Classification results

Datasets have been split in training (60%), validation (20%) and test sets (20%) in a stratified manner in order to perform the training and performance evaluation. For our evaluations, we used the most common metrics for classification tasks Pendleton et al. (2016): i) *Accuracy*, which is the correct predictions ratio; ii) *Precision*, which measures the proportion of truly malicious API calls among all the detected samples; iii) *Recall*, which quantifies the number of detected malicious API calls among all of them, quantifying the false negative rate; iv) *F1 score*, which measures the overall effectiveness by combining precision and recall.

Table 4 reports the performance for each dataset and model we tested (refer to the Appendix for detailed metrics for each class). The (MultiHead) attention performance refers to a different model, trained with the (multi-head) attention layer after the LSTM.

Specifically, the LSTM achieves better results, in terms of *F1 score*, than GRU and BiLSTM on two out of three datasets. Consequently, we chose this model as the one to be explained by post-hoc XAI techniques, i.e. LIME, SHAP and LRP.

Interestingly, the standard soft attention mechanism does not improve too much the classification performance, except for the *Mal-API-2019* dataset where we gain up to 6 points over the LSTM base model. In addition, multi-head attention seems not beneficent since it always leads to worse metrics. This situation can be also found in (Wiegrefe and Pinter (2019)), where the authors deemed the datasets unsuitable test cases to evaluate explanations, claiming that “attention is not ex-

planation if you don't need it”. However, we decided to keep the soft attention mechanism for the explanation-related experiments so as to empirically validate its suitability in this domain. Finally, it is worth to note that we did not observe significant performance differences when increasing the sequence length parameter over the median for each dataset (see Table 3), meaning that the truncation pre-processing strategy does not lose malicious API (sub-)sequences.

5.3. Efficiency analysis

When performing malware detection, analysts should understand what is happening in the system as soon as possible, before the malware makes more damages or spreads across the network. From this perspective, the explanation could speed up the process if its generation is time efficient; in other words, even if the explanation could help the analysts in understanding what has happened, it is useless if its generation requires too much time.

To this end, we evaluate the running times of the analyzed explainers for each dataset. It is worth to note that for post-hoc techniques (LRP, LIME and SHAP) the explanation time must be added to the prediction one.

Table 5 reports the mean run times and the 95% confidence intervals computed considering the instances of the testing sets for each dataset (1,500 samples for Alibaba, 1,422 samples for Mal-API-2019 and 8,776 samples for API Call Sequences, respectively). Not surprisingly, LRP achieves by far the best results across all datasets, 0.12, 0.17 and 0.99 seconds respectively. The reason behind this performance is explained by the fact that it just requires a single back-propagation step along the network model, in contrast to other explainers which require fitting either a statistical model (SHAP) or a transparent machine learning algorithm (LIME). In any case, the running times on the Alibaba dataset (0.99 seconds) are quite worse because of the longer input API sequences.

The Attention running times are not reported since attention mechanism is included into the network, so the explanations are computed simultaneously with the prediction. Interestingly, SHAP performance strongly depends on the instances on which the explainer is built. To perform the experiment, we set the maximum number of training samples that our machine could handle.

5.4. Effectiveness analysis

5.4.1. Compatibility

The experimented explanation methods rely on different assumptions depending on what they “know” about the model and/or the data it is trained on. Thus, we first assess the compatibility degree between different explanations: specifically, we compared the set T_i of the k features with the highest relevance from method i with the set T_j of the k features with the highest relevance from method j , for each method i, j . Formally,

$$IS(T_i, T_j) = \frac{|T_i \cap T_j|}{k}, \quad (2)$$

i, j being the four techniques we analyzed. We set k as the 10% of the sequence length. Fig. 4 shows that there is little agreement, lower than 50%, between different techniques, especially on GRU explanations. Thus, this highlights the need of a comprehensive comparison to understand which are the best explanations and in which situations.

5.4.2. Sequence-level representations

A qualitative analysis has been conducted in order to explore if the word relevance can be leveraged to build sequence-level representations that present some regularities. The motivation behind this approach is to investigate whether the relevance of words can be harnessed to enhance the explainability of sequence-level representations.

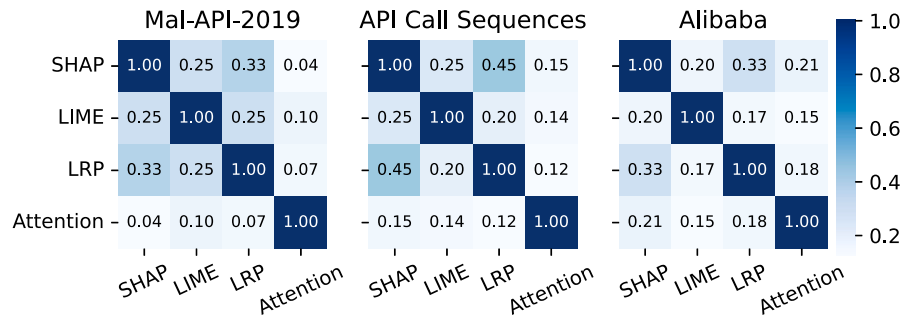
⁹ <https://github.com/slundberg/shap>.

¹⁰ https://github.com/ArrasL/LRP_for_LSTM.

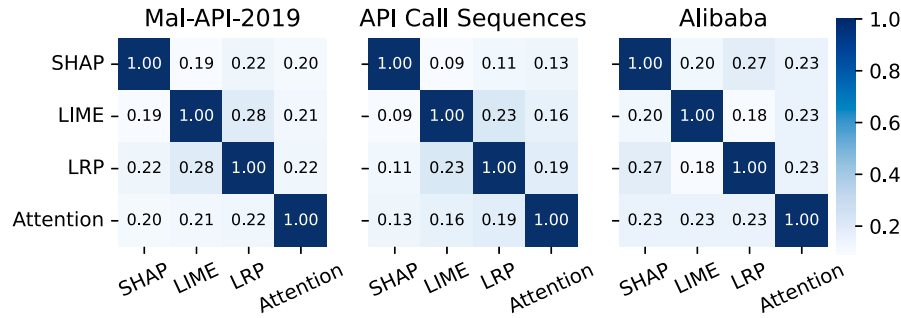
Table 4

Classification results for each dataset and model. The two *attention*-based models are built considering the respective attention layer after the LSTM one.

Dataset	Model	Accuracy	Precision	Recall	F1
Mal-API-2019	LSTM	47.53	49.62	47.82	48.47
	BiLSTM	52.88	54.89	53.99	54.31
	GRU	47.69	49.10	48.62	48.27
	Attention	52.18	53.92	53.32	53.40
	MultiHeadAttention	47.69	49.10	48.62	48.27
API Call Sequences	LSTM	99.43	95.69	91.81	93.79
	BiLSTM	98.91	93.86	81.86	86.85
	GRU	98.91	91.62	84.30	87.49
	Attention	99.32	94.59	91.08	92.71
	MultiHeadAttention	98.99	92.87	84.92	88.38
Alibaba	LSTM	83.69	66.60	67.54	67.05
	BiLSTM	82.90	67.21	62.88	63.55
	GRU	84.09	72.02	65.20	65.81
	Attention	85.71	69.51	67.78	68.32
	MultiHeadAttention	84.72	67.23	66.85	66.47



(a) LSTM



(b) GRU

Fig. 4. Compatibility between different explanation techniques.**Table 5**

Explanation Run times (s) over the three datasets.

Method	Mal-API-2019	API Call Sequences	Alibaba
LIME	0.71 ± 0.001	0.59 ± 0.0005	2.52 ± 0.002
SHAP	0.28 ± 0.002	1.59 ± 0.0005	2.23 ± 0.006
LRP	0.12 ± 0.002	0.17 ± 0.0001	0.99 ± 0.001
Attention	—	—	—

By visualizing these representations in a 2D space, we aim to gain insights into the separability of different classes and the effectiveness of different explanation methods.

Specifically, the word embeddings are obtained from the output of the Embedding layer in our model. The construction process involves a linear combination of these word embeddings. The weighting factor

for each word embedding in this combination is its corresponding *importance* score, as provided by the explanation method. This approach is based on the hypothesis that relevant words contribute more significantly to the overall context of the sequence. For each explanation method, we select the variant that yielded the best performance in our previous experiments. We also include results from a baseline method, which involves simple averaging of word embeddings without considering word relevance.

The sequence-level representations are then projected onto a 2D space using Principal Component Analysis (PCA). This step is performed to visualize the distribution and separation of sequences in a lower-dimensional space. The PCA is applied after whitening the data to ensure each principal component has a unit variance.

The resulting 2D projections for the test set sequences are depicted in Fig. 5 and Fig. 6 for the LSTM and GRU models, respectively. It is

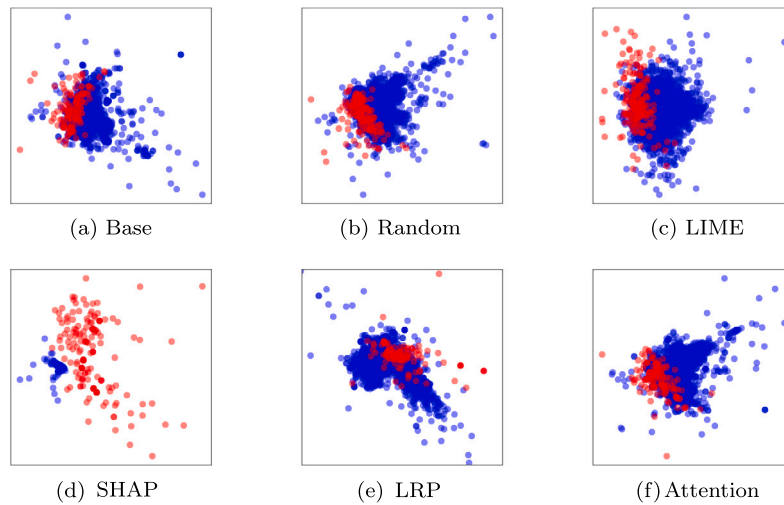


Fig. 5. Sequence-level representations of the LSTM model for the Api Call Sequences dataset. Red points represent malware instances, while blue ones represent goodware instances.

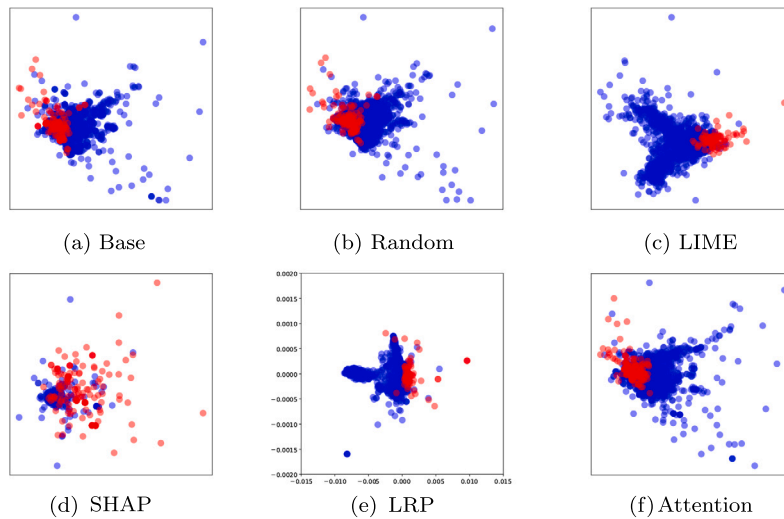


Fig. 6. Sequence-level representations of the GRU model for the Api Call Sequences dataset. Red points represent malware instances, while blue ones represent goodware instances.

noteworthy that the SHAP explanation method exhibits superior space-separation between malware and goodware instances for the LSTM model. However, this observation does not hold for the GRU model, where a high degree of overlap between the two classes is observed.

5.4.3. Perturbation experiment

We perform a *perturbation experiment*, a.k.a. “pixel-flipping” in computer vision (Samek et al. (2016)), by perturbing system calls from the input sequences according to their relevance values, and tracking the impact on the classification performance (Qiu et al. (2022)). This quantitative analysis aims to answer two questions:

- were the features extracted from the explanation the ones the model looked at for its prediction? (*perturbation experiment*)
- are the most important features provided by explanations sufficient to make the model do its prediction? (*sufficiency experiment*)

In particular, we consider three token perturbation strategies Zhang et al. (2020): (1) removal, (2) random modification and (3) replacement with the padding token. We perturb a percentage of the APIs per sequence considering the ascending/descending order of the scores assigned by explainers. Specifically, we perform two experiments: i)

perturbation of the 1%, 5% and 10% of the most relevant APIs (perturbation experiment); ii) perturbation of the 70%, 80%, 90%, 95%, 99% of the least relevant APIs (sufficiency experiment).

In Table 6 (resp. Table 7) we report the mean changes in LSTM (resp. GRU) prediction probabilities when perturbing the 1% of the most important tokens (i.e., perturbation experiment) and the 99% of the least important tokens (i.e., sufficiency experiment). What stands out is that the perturbation strategy (i.e. removal, random modification, replacement) can have strong influence on the changes in prediction probabilities, but the comparison of explanation techniques is not influenced by such differences.

Fig. 7 (a) and (b) show the full results of the perturbation experiment, for LSTM and GRU models, respectively. Each plot reports the mean changes in prediction probabilities varying the percentage of perturbed instances. Results are averaged across the three perturbation types. More specifically, in Fig. 7(a) the percentage of perturbed tokens varies from 1% to 10% and SHAP and LRP provide by far the most comprehensive explanations since there is a significant drop of the target class probability even removing a small percentage of the top rated APIs (e.g. about 40% drop by removing the 1% top calls rated by SHAP on the Alibaba dataset). LIME performance is generally worse than SHAP and LRP, due to its uncertainty (Zhang et al. (2019)) and

Table 6

Effects of the tokens perturbation strategy on the perturbation experiment for the LSTM model. Results refer to perturbing the 1% (99%) of the most (least) relevant API calls for the perturbation (sufficiency) experiment.

Dataset	Explainer	Perturbation			Sufficiency		
		Delete	Replace	Padding	Delete	Replace	Padding
Mal-API-2019	Random	-4.39 ± 0.32	-3.15 ± 0.28	-1.00 ± 0.15	-47.18 ± 1.291	-26.55 ± 1.773	-44.74 ± 2.020
	LIME	-7.33 ± 0.42	-6.12 ± 0.40	-4.10 ± 0.32	-46.49 ± 1.267	-21.64 ± 1.646	-44.68 ± 2.042
	LRP	-6.78 ± 0.40	-4.91 ± 0.36	-3.61 ± 0.29	-46.28 ± 1.268	-19.08 ± 1.607	-44.69 ± 2.025
	SHAP	-6.36 ± 0.38	-4.94 ± 0.36	-3.09 ± 0.28	-46.48 ± 1.262	-18.95 ± 1.615	-44.71 ± 2.022
	Attention	0.23 ± 5.79	-0.95 ± 8.38	0.30 ± 5.90	-12.69 ± 1.698	-10.28 ± 1.461	-12.78 ± 1.693
API Call Sequences	Random	-0.88 ± 0.17	-0.83 ± 0.17	-0.37 ± 0.11	-34.03 ± 0.670	-13.54 ± 0.654	-22.59 ± 0.337
	LIME	-1.49 ± 0.22	-1.66 ± 0.23	-1.31 ± 0.20	-17.59 ± 0.602	-12.56 ± 0.634	-19.58 ± 0.422
	LRP	-14.43 ± 0.66	-12.07 ± 0.63	-13.39 ± 0.67	-6.815 ± 0.466	-12.03 ± 0.618	-22.95 ± 0.503
	SHAP	-19.01 ± 0.75	-11.99 ± 0.63	-13.76 ± 0.68	-2.272 ± 0.273	-10.98 ± 0.598	-12.46 ± 0.378
	Attention	0 ± 0	0 ± 0	0 ± 0	-19.82 ± 0.912	-8.719 ± 0.622	1.675 ± 0.301
Alibaba	Random	-2.92 ± 0.57	-6.93 ± 0.81	-0.76 ± 0.25	-64.67 ± 1.292	-55.38 ± 1.560	-75.09 ± 0.912
	LIME	-13.83 ± 1.16	-20.53 ± 1.34	-12.25 ± 1.09	-52.37 ± 1.536	-43.87 ± 1.600	-60.44 ± 1.439
	LRP	-23.95 ± 1.38	-34.18 ± 1.55	-23.57 ± 1.35	-49.73 ± 1.563	-36.15 ± 1.568	-46.12 ± 1.598
	SHAP	-38.59 ± 1.58	-46.23 ± 1.60	-42.56 ± 1.57	-40.49 ± 1.529	-27.68 ± 1.467	-32.68 ± 1.533
	Attention	-0.53 ± 11.18	-5.42 ± 25.48	0.02 ± 11.22	-46.39 ± 1.800	-34.29 ± 1.753	-32.05 ± 2.009

Table 7

Effects of the tokens perturbation strategy on the perturbation experiment for the GRU model. Results refer to perturbing the 1% (99%) of the most (least) relevant API calls for the perturbation (sufficiency) experiment.

Dataset	Explainer	Perturbation			Sufficiency		
		Delete	Replace	Padding	Delete	Replace	Padding
Mal-API-2019	Random	-1.20 ± 0.19	-2.04 ± 0.46	-0.24 ± 0.15	-40.38 ± 35.55	-40.36 ± 35.52	-37.24 ± 29.79
	LIME	-5.75 ± 0.65	-7.52 ± 0.82	-5.12 ± 0.63	-40.38 ± 35.55	-37.64 ± 35.21	-33.79 ± 30.08
	LRP	-8.50 ± 0.75	-10.61 ± 0.89	-8.47 ± 0.74	-40.38 ± 35.55	-36.44 ± 34.91	-32.11 ± 30.01
	SHAP	-4.13 ± 0.58	-5.76 ± 0.77	-3.35 ± 0.57	-40.38 ± 35.55	-38.93 ± 35.32	-34.55 ± 29.62
	Attention	-0.78 ± 0.25	-2.56 ± 0.48	-0.85 ± 0.26	-55.54 ± 40.90	-55.67 ± 40.87	-44.04 ± 25.53
API Call Sequences	Random	-0.92 ± 0.13	-0.98 ± 0.17	-0.22 ± 0.07	-42.77 ± 34.24	-24.11 ± 12.84	-3.069 ± 16.53
	LIME	-2.49 ± 0.32	-2.37 ± 0.36	-1.16 ± 0.22	-38.91 ± 36.71	-14.11 ± 17.74	-2.925 ± 16.48
	LRP	-7.09 ± 0.47	-7.45 ± 0.48	-6.08 ± 0.44	-27.28 ± 34.03	-14.75 ± 18.80	-2.777 ± 16.02
	SHAP	-0.78 ± 0.14	-1.19 ± 0.21	-0.47 ± 0.14	-26.89 ± 25.08	-30.02 ± 24.44	-3.264 ± 16.98
	Attention	-4.93 ± 0.31	-4.38 ± 0.29	-4.59 ± 0.29	-45.37 ± 34.65	-21.43 ± 16.97	-2.881 ± 16.19
Alibaba	Random	-0.20 ± 0.06	-1.66 ± 0.30	-0.07 ± 0.04	-40.43 ± 38.71	-44.90 ± 41.86	-37.93 ± 36.56
	LIME	-2.15 ± 0.36	-4.61 ± 0.53	-1.98 ± 0.34	-32.26 ± 35.15	-41.19 ± 41.34	-37.81 ± 36.47
	LRP	-15.29 ± 0.94	-21.46 ± 1.11	-17.08 ± 0.98	-7.910 ± 19.87	-12.76 ± 25.46	-33.43 ± 34.83
	SHAP	-10.59 ± 1.51	-11.81 ± 1.55	-10.85 ± 1.55	-23.48 ± 29.66	-19.83 ± 30.50	-37.43 ± 35.90
	Attention	-0.89 ± 0.23	-3.92 ± 0.53	-1.34 ± 0.30	-35.99 ± 38.79	-39.42 ± 37.35	-39.42 ± 37.46

reliance on datasets and models involved in the explanation (Nguyen (2018)). On the other hand, for the GRU model LRP method proves to achieve the best performance for all the datasets under analysis.

Furthermore, the results of the sufficiency experiment are illustrated in Fig. 8 (a) and (b) for the LSTM and GRU models, respectively. Each plot depicts the variations in prediction probabilities upon perturbing a percentage of the least important API calls, thereby preserving only a portion of unchanged tokens (ranging from 1% to 30%). For example, the LSTM's predictions do not change at all for the API Call Sequences dataset when removing 70% of the least important features identified through SHAP. In general, the results once again underscore the dominance of SHAP for the LSTM model and LRP for the GRU model when evaluating the sufficiency of the provided explanations. This discrepancy can be attributed to the complexity contrast between the models under analysis, where the LSTM outperforms the GRU (as outlined in Table 4) by learning more intricate representations and incorporating additional parameters. As a result, a gradient-based method like LRP could more effortlessly propagate relevance through the layers of simpler models (like GRU), further emphasizing the intricate interplay between model complexity and the effectiveness of explanation methods.

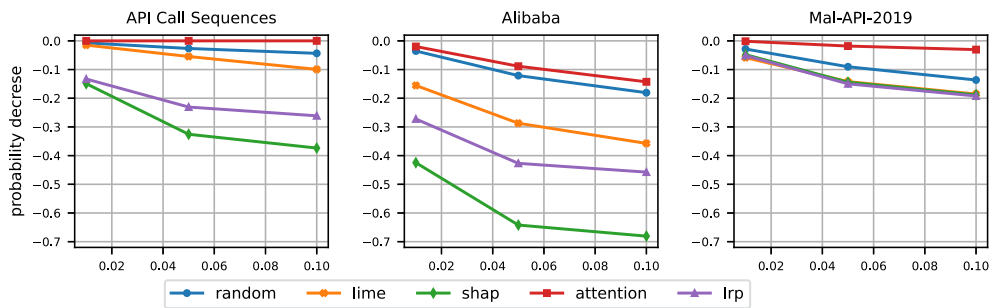
With regards to the attention mechanism, in the perturbation experiment we can observe that it seems to not have caught the real patterns the model looks at, when making predictions. These results are con-

sistent with the current literature (Jain and Wallace (2019); Wiegrefe and Pinter (2019); Schlegel and Keim (2023)) which discusses about the low explainability power of the attention mechanism in various NLP tasks (e.g. text classification, natural language inference) and time series analysis, respectively. However, when considering the sufficiency experiment, the attention is competitive with other techniques and, in the case Mal-API-2019 dataset, it even provides the best explanations because predictions do not change when perturbing useless features.

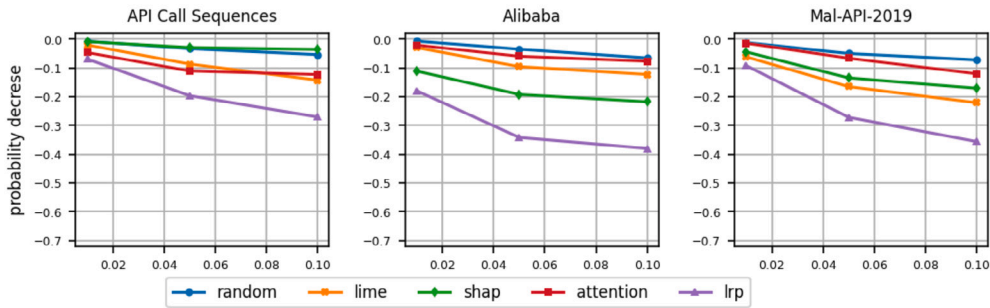
5.4.4. Stability

The explanations provided for the security domain should be reliable in order to be considered by expert analysts. In other words, the scores representing the system calls importance need to be stable over time: we expect homogeneous explanations for the same sequence or similar ones. Besides, from the XAI perspective, unstable explanations cannot express the real knowledge of the model.

We measured stability of the XAI techniques explaining the same instances multiple times and measuring the intersection of explanations as defined in equation (2). Specifically, we defined the explainer with 20% of the training dataset, repeated explanations 30 times and averaged the results over 100 sequences from the testing set, 95% confidence intervals have been computed as well. Table 8 shows that LIME cannot provide stable explanations for any dataset and model, achieving very low scores, limiting its applicability to real scenarios. This behavior

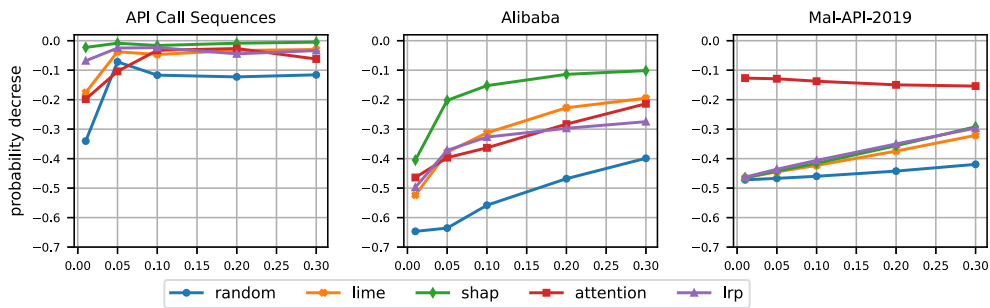


(a) LSTM

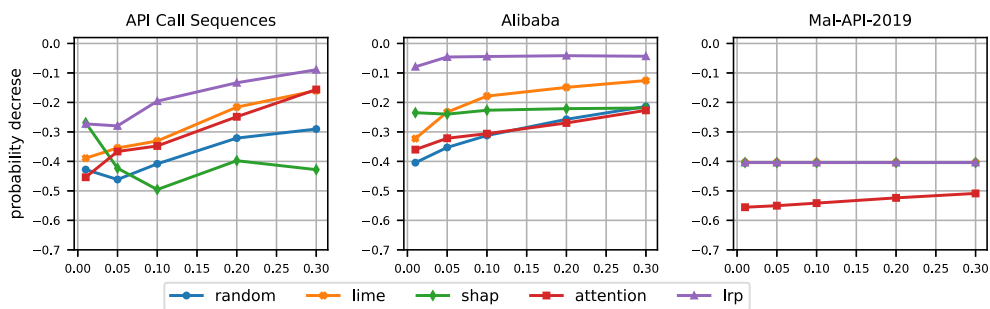


(b) GRU

Fig. 7. Results showing how the probability of the predicted class changes when perturbing the most relevant features for the LSTM (a) and GRU (b) models.



(a) LSTM



(b) GRU

Fig. 8. Results showing how the probability of the predicted class changes when perturbing least relevant features for the LSTM (a) and GRU (b) models.

Table 8
Stability results.

Method	LSTM			GRU		
	Mal-API-2019	API Call Sequences	Alibaba	Mal-API-2019	API Call Sequences	Alibaba
LIME	0.1505 ± 0.0137	0.0806 ± 0.0091	0.0914 ± 0.0072	0.1376 ± 0.0112	0.1155 ± 0.0114	0.0940 ± 0.0064
SHAP	0.8076 ± 0.0842	0.7984 ± 0.0914	0.4995 ± 0.0292	0.5701 ± 0.0444	0.5920 ± 0.0380	0.3095 ± 0.0304
LRP	1.000	1.000	1.000	1.000	1.000	1.000
Attention	1.000	1.000	1.000	1.000	1.000	1.000

clearly depends on the variability introduced by the stochastic neighbors' generation process, i.e. if the neighborhood is highly unstable, the explanation's locality changes as well. On the other hand, SHAP reaches good results on two out of the three datasets, especially on the LSTM model. Specifically, the poorer stability on the Alibaba dataset probably depends on the longer malware sequences which, in turn, determine greater errors when approximating Shapley values. Finally, LRP and Attention provide, by construction, deterministic explanations thus reaching the maximum stability score.

5.5. Case study

We conduct a case study on the different malware classes of the Alibaba dataset for showing how XAI analysis can support practitioners in the understanding of malicious behaviors. Fig. 9 shows the LRP scores assigned to API call sequences classified as *Backdoor*, *Downloader*, and *Trojan*, respectively.

- Fig. 9a (*Backdoor*) This attack aims at maintaining a connection with a remote host bypassing any network security mechanism and does not perform any peculiar malicious activity. The explanation unveils this behavior due to the high relevance assigned to the *InternetOpenA* and *InternetOpenUrlA* calls, meaning that the malware is trying to connect via URL. On the contrary, the other API calls which represent the actual activity performed on the remote host (e.g. *CreateService* and *FindResource*) are not considered important for the classification;
- Fig. 9b (*Downloader*). A Downloader is a type of Trojan horse that downloads and installs malicious software without the consent of the user. The explanation provided by LRP highlights the *NtOpenDirectoryObject* call which tries to copy files from a part to another of the file system.¹¹
- Fig. 9c (*Trojan*). A Trojan Horse malware downloads onto a computer disguised as a legitimate program. Usually, the attacker hides malicious code within a legitimate software to gain access to the system of a user. In figure, we can notice that *NtFreeVirtualMemory* and *NtAllocateVirtualMemory* calls have been assigned a high score, probably due to a buffer overflow tentative which is trying to work with virtual memory in order to inject commands. Furthermore, *GetSystemTimeAsFileTime* is a well-known part of malicious Trojan softwares.¹²

Specifically, this approach reduces the number of false positives by enabling a Security Operation Center (SOC) operator, for example, to focus on specific patterns, as also discussed in Chen et al. (2024). For instance, the API call *CreateProcessInternalW*, which has been requested by the service A (*CreateServiceA*) installed through the use of the backdoor (*InternetOpenA*), initiates the execution of a process that can be monitored through a hidden interface. The emphasis placed on the API call *NtFreeVirtualMemory* focuses the operator's attention on previous processes that have allocated memory for instantiating a service. In addition, the malware analyst can more quickly unveil distinctive patterns

to identify the attack techniques used by the specific malware. In fact, the same evidence was found within numerous procedures of MITRE within the "Command and Control" tactic¹³ referred to the "Application Layer Protocol: Web Protocols" technique,¹⁴ which inherently involves the API call *InternetOpenA*.

5.6. Discussion

Looking at the big picture, the data-driven nature of the CTI process can be strongly improved with AI-based systems. Nevertheless, XAI methods and tools are essential to make the most recent AI advancements applicable in real security scenarios, thus enhancing the need of jointly integrating the two elements into existing CTI platforms. From the end-user viewpoint, the idea is to replicate the same thinking process a domain expert does when reaching a conclusion supported by the results of (a series of) statistical methods. From this perspective, the explanation should reveal those insights or at least guide the analyst towards them. For instance, when discovering the system's components affected by a malware, the explanations could support the analyst in understanding what has happened, why it happened and which interventions to prioritize to restore the system.

The experiments conducted in the previous sections highlight that further research is needed in the XAI field to achieve these goals. Firstly, the compatibility study shows that there is little accordance between XAI techniques, thus increasing the need to define ground truth explanations for predictions.

Meanwhile, the explanation quality can be estimated indirectly by measuring how explanation-driven experiments impact on model performance. From this perspective, SHAP established its superiority in both the perturbation and stability experiments but its limitation is clear when considering the efficiency: not only is the computation time higher w.r.t. other techniques, but also this gap increases exponentially when considering more (training) data or longer sequences. On the other hand, LRP performs slightly worse on the perturbation experiment but outperforms all other post-hoc methods when it comes to stability and time efficiency.

In general, LIME performs worse than SHAP and LRP and reveals its greatest limitation when considering the explanations' stability: on two out of the three datasets we considered, the method is able to retrieve repeatedly less than the 10% of the top-ranked explanations.

The sequence-level representations obtained by scaling each API call embedding with the score assigned by the explainer before averaging all the embeddings, show that SHAP delivers the best structured representation, while LRP represents all the *malware* instances in a dense region.

Finally, the attention mechanism seems the worst technique for at least three reasons. First, even if it increases model complexity, it does not improve classification performance. Second, it requires model re-training and thus cannot be used with off-the-shelf models. Third, from the XAI perspective, even though the attention scores are consistent for not relevant features, they do not find out the most important patterns leading to the actual predictions. In complete fairness, we can also outline that our dataset might not be suited to understand whether

¹¹ <https://www.malwarebytes.com/blog/news/2017/05/the-worm-that-spreads-wanacrypt0r>.

¹² <https://vms.drweb-av.it/virus/?i=3843863>.

¹³ <https://attack.mitre.org/tactics/TA0011/>.

¹⁴ <https://attack.mitre.org/techniques/T1071/001/>.

NtDuplicateObject InternetOpenA InternetOpenUrlA InternetCloseHandle OpenSCManagerA CreateServiceA StartServiceA LdrGetDllHandle LdrGetProcedureAddress FindResourceA LoadResource
SizeofResource NtOpenFile NtClose NtCreateFile NtWriteFile NtClose CreateProcessInternalW LdrGetDllHandle NtTerminateProcess NtClose NtUnmapViewOfSection NtClose NtFreeVirtualMemory NtClose
LdrGetProcedureAddress NtClose NtUnmapViewOfSection NtClose NtUnmapViewOfSection NtClose RegCloseKey NtClose LdrGetProcedureAddress NtUnmapViewOfSection NtClose RegCloseKey NtClose
LdrUnloadDll NtOpenKey NtQueryValueKey NtClose LdrUnloadDll NtClose NtTerminateProcess LdrGetProcedureAddress ReadProcessMemory LdrLoadDll LdrUnloadDll LdrGetProcedureAddress LdrLoadDll
LdrGetProcedureAddress ReadProcessMemory LdrLoadDll LdrGetProcedureAddress NtAllocateVirtualMemory RegOpenKeyExA RegCreateKeyExA RegCloseKey RegCreateKeyExA RegSetValueExA
RegQueryValueExA LdrGetProcedureAddress LdrGetDllHandle CreateThread RegQueryValueExA LdrLoadDll LdrGetProcedureAddress OpenSCManagerA LdrGetProcedureAddress OpenServiceA

(a) Backdoor

NtOpenDirectoryObject LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle NtClose GetSystemTimeAsFileTime SetUnhandledExceptionFilter LdrGetDllHandle NtDuplicateObject
NtAllocateVirtualMemory WSASStartup GetSystemTimeAsFileTime GetSystemInfo SetErrorMode LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle LdrGetProcedureAddress RegOpenKeyExW
RegQueryValueExW RegCloseKey RegOpenKeyExW RegQueryValueExW RegCloseKey GetSystemInfo LdrGetDllHandle LdrGetProcedureAddress RegOpenKeyExW GetSystemDirectoryW RegOpenKeyExA
GetSystemWindowsDirectoryW LoadStringW GetSystemWindowsDirectoryW GetSystemDirectoryW RegOpenKeyExW LdrLoadDll LdrGetProcedureAddress RegQueryValueExW RegCloseKey RegOpenKeyExW
RegQueryValueExW RegCloseKey NtCreateMutant GetNativeSystemInfo GetSystemWindowsDirectoryW NtClose LdrLoadDll LdrGetProcedureAddress WSASStartup socket NtQueryAttributesFile NtCreateFile

(b) Downloader

GetSystemTimeAsFileTime NtAllocateVirtualMemory NtFreeVirtualMemory NtAllocateVirtualMemory LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle
LdrGetProcedureAddress LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle LdrGetProcedureAddress
LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle LdrGetProcedureAddress LdrGetDllHandle LdrGetProcedureAddress NtAllocateVirtualMemory LdrGetDllHandle
LdrGetProcedureAddress SetUnhandledExceptionFilter CoInitializeEx WSASStartup NtAllocateVirtualMemory LdrGetDllHandle NtOpenFile NtClose RegOpenKeyExW RegCreateKeyExW RegDeleteValueW
RegSetValueExW RegCloseKey NtAllocateVirtualMemory NtClose RegCloseKey NtClose RegCloseKey NtClose CoInitialize LdrGetDllHandle NtTerminateProcess NtClose LdrUnloadDll NtOpenKey

(c) Trojan

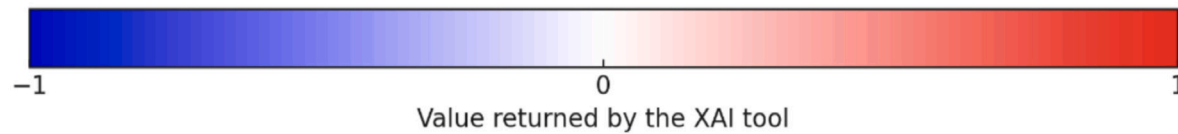


Fig. 9. LRP scores assigned to API call sequences classified as *Backdoor* (a), *Downloader* (b) and *Trojan* (c). Red and blue tokens have a positive and negative influence to prediction, respectively. The color intensity indicates the importance of the token as returned by LRP.

or not the attention mechanism provides explanations since its learning power is not exploited during the learning process (Wiegrefe and Pinter (2019)).

6. Conclusions & future work

In this paper, we have proposed an eXplainable Artificial Intelligence framework for the behavioral malware detection problem. The main goal has been to evaluate the effectiveness of four XAI methods applied to a sequence-based deep learning architecture, and their applicability and suitability in the modern CS scenarios.

In particular, the reported experiments have shown, as highlighted in our discussions, the feasibility of different XAI techniques for explaining the decision of the LSTM-based classifier both in terms of explanations' quality and running times necessary to obtain the related results. Furthermore, other experiments have proven the stability and flexibility of the exploited tools. Note that we analyze *local* explanation methods, i.e. methods in this work are able to explain predictions about single input samples. *Global* explanations are out of the scope of this work.

However, one of the main limitations of this analysis mainly concerns the effectiveness evaluation of XAI methodologies due to the lack of qualitative metrics to directly evaluate the effectiveness of XAI methodologies. Another limitation may be related to the analyzed domain, that could affect the main findings of the analysis.

Future work will be devoted to test additional XAI methods and the robustness of the proposed architecture with respect to adversarial attacks and how such kind of perturbations can influence the quality of produced explanations. Furthermore, we wonder whether the explanations can be exploited to improve the classification performance even supporting the discovery of systematic errors of the predictive model. Finally, we will evaluate the explanations in real-world scenarios measuring their effectiveness and efficacy w.r.t. how they improve or speed up the expert analyst's work.

CRedit authorship contribution statement

Antonio Galli: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Valerio La Gatta:** Formal analysis, Data curation, Conceptualization, Methodology, Software, Validation, Writing – original draft, Writing – review & editing. **Vincenzo Moscato:** Conceptualization, Data curation, Formal analysis, Methodology, Software, Validation, Writing – original draft, Writing – review & editing. **Marco Postiglione:** Conceptualization, Data curation, Formal analysis, Methodology, Software, Validation, Writing – original draft, Writing – review & editing. **Giancarlo Sperli:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Conceptualization, Data curation, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Appendix A. Classification results

Tables 9, 10 and 11 show the category-wise results obtained by the black-box LSTM model in terms of Precision, Recall and F1 for *Mal-API-2019*, *Alibaba* and *API Call Sequences* datasets, respectively.

Appendix B. PCA experiment

Fig. 10 and 11 show the resulting 2D whitened PCA projections of the test set sequences for Alibaba and Catak datasets.

References

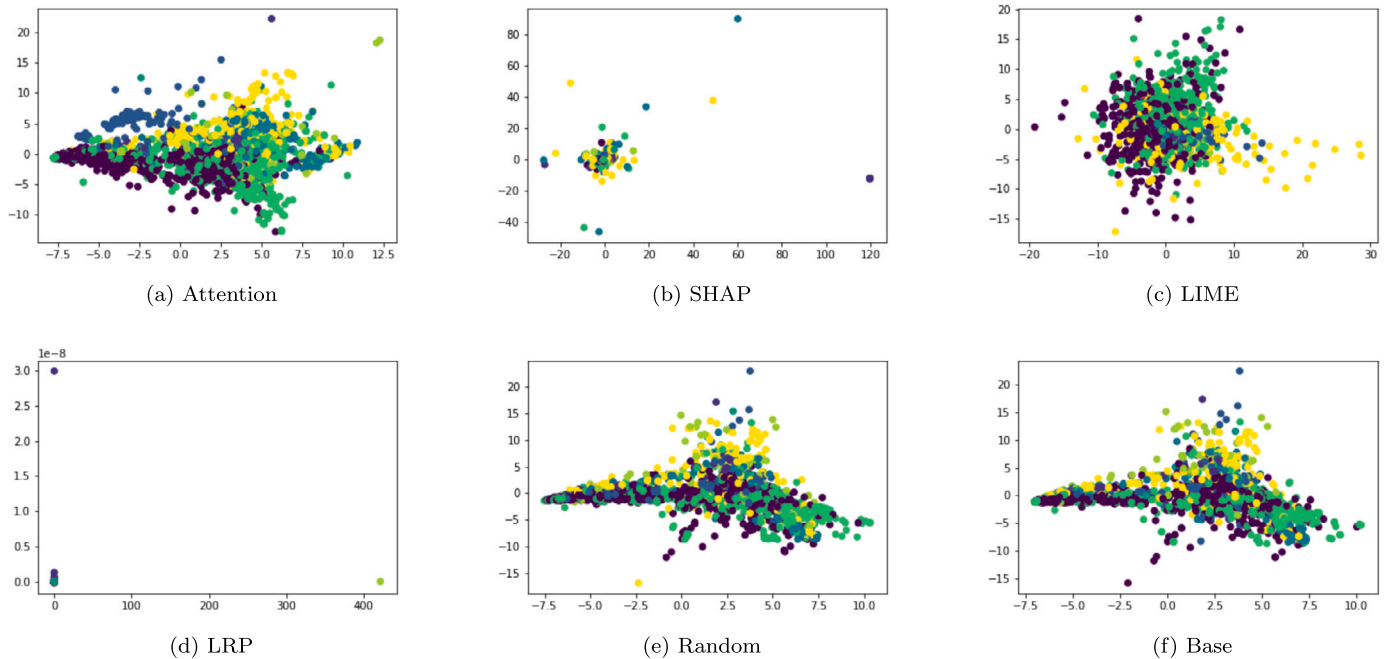
- Alazab, M., Layton, R., Venkatraman, S., Watters, P., 2010. Malware detection based on structural and behavioural features of api calls. In: *International Cyber Resilience Conference (1st: 2010)*. Edith Cowan University, pp. 1–10.
- Ali, S., Abuhmed, T., El-Sappagh, S., Muhammad, K., Alonso-Moral, J.M., Confalonieri, R., Guidotti, R., Del Ser, J., Díaz-Rodríguez, N., Herrera, F., 2023. Explainable artificial intelligence (XAI): what we know and what is left to attain trustworthy artificial intelligence. *Inf. Fusion* 99, 101805. <https://doi.org/10.1016/j.inffus.2023.101805>.
- Alibaba Cloud, 2018. Alibaba cloud malware detection based on behaviors. <https://tianchi.aliyun.com/getStart/information.htm?raceId=231694>. (Accessed 2 January 2024).
- Alvarez-Melis, D., Jaakkola, T., 2018. On the robustness of interpretability methods. *arXiv:1806.08049 [abs]*.
- Arras, L., Montavon, G., Müller, K.R., Samek, W., 2017. Explaining recurrent neural network predictions in sentiment analysis. *EMNLP 2017*, 159.
- Arras, L., Osman, A., Müller, K.R., Samek, W., 2019. Evaluating recurrent neural network explanations. Preprint. *arXiv:1904.11829*.
- Aversano, L., Bernardi, M.L., Cimitile, M., Iammarino, M., Verdone, C., 2023. A data-aware explainable deep learning approach for next activity prediction. *Eng. Appl. Artif. Intell.* 126, 106758. <https://doi.org/10.1016/j.engappai.2023.106758>.
- Bach, S., Binder, A., Montavon, G., Klauschen, F., Müller, K., Samek, W., 2015. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLoS ONE* 10. <https://doi.org/10.1371/journal.pone.0130140>.
- Bahdanau, D., Cho, K.H., Bengio, Y., 2015. Neural machine translation by jointly learning to align and translate. In: *3rd International Conference on Learning Representations, ICLR 2015*.
- Barata, C., Celebi, M.E., Marques, J.S., 2021. Explainable skin lesion diagnosis using taxonomies. *Pattern Recognit.* 110, 107413. <https://doi.org/10.1016/j.patcog.2020.107413>.
- Bazrafshan, Z., Hashemi, H., Fard, S.M.H., Hamzeh, A., 2013. A survey on heuristic malware detection techniques. In: *The 5th Conference on Information and Knowledge Technology, IEEE*, pp. 113–120.
- Bhat, P., Behal, S., Dutta, K., 2023. A system call-based Android malware detection approach with homogeneous & heterogeneous ensemble machine learning. *Comput. Secur.* 130, 103277. <https://doi.org/10.1016/j.cose.2023.103277>.
- Catak, F.O., Yazı, A.F., Elezaj, O., Ahmed, J., 2020. Deep learning based sequential model for malware analysis using windows exe api calls. *PeerJ Comput. Sci.* 6, e285. <https://doi.org/10.7717/peerj-cs.285>.
- Cavaliere, D., Gallo, M., Stanzione, C., 2023. Propaganda detection robustness through adversarial attacks driven by eXplainable AI. In: *World Conference on Explainable Artificial Intelligence*. Springer, pp. 405–419.
- Chakraborty, T., Pierazzi, F., Subrahmanian, V.S., 2020. Ec2: Ensemble clustering and classification for predicting Android malware families. *IEEE Trans. Dependable Secure Comput.* 17 (2), 262–277. <https://doi.org/10.1109/TDSC.2017.2739145>.
- Chen, B., Liu, Y., Zhang, Z., Lu, G., Kong, A.W.K., 2023. TransAttUnet: multi-level attention-guided U-net with transformer for medical image segmentation. *IEEE Trans. Emerg. Top. Comput. Intell.*, 1–14. <https://doi.org/10.1109/TETCI.2023.3309626>.
- Chen, T., Zeng, H., Lv, M., Zhu, T., 2024. CTIMD: Cyber threat intelligence enhanced malware detection using API call sequences with parameters. *Comput. Secur.* 136, 103518. <https://doi.org/10.1016/j.cose.2023.103518>.
- Cheng, B., Ming, J., Leal, E.A., Zhang, H., Fu, J., Peng, G., Marion, J.Y., 2021. {Obfuscation-Resilient} executable payload extraction from packed malware. In: *30th USENIX Security Symposium (USENIX Security, vol. 21, pp. 3451–3468*.
- Damodaran, A., Di Troia, F., Visaggio, C.A., Austin, T.H., Stamp, M., 2017. A comparison of static, dynamic, and hybrid analysis for malware detection. *J. Comput. Virol. Hacking Tech.* 13 (1), 1–12. <https://doi.org/10.1007/s11416-015-0261-z>.
- Demirkiran, F., Çayır, A., Ünal, U., Dağ, H., 2022. An ensemble of pre-trained transformer models for imbalanced multiclass malware classification. *Comput. Secur.* 121, 102846. <https://doi.org/10.1016/j.cose.2022.102846>.
- Du, M., Liu, N., Hu, X., 2019. Techniques for interpretable machine learning. *Commun. ACM* 63 (1), 68–77. <https://doi.org/10.1145/3359786>.
- Dwarampudi, M., Reddy, N.V.S., 2019. Effects of padding on lstms and cnns. *CoRR*. *arXiv:1903.07288*.
- Fan, M., Wei, W., Xie, X., Liu, Y., Guan, X., Liu, T., 2021. Can we trust your explanations? Sanity checks for interpreters in Android malware analysis. *IEEE Trans. Inf. Forensics Secur.* 16, 838–853. <https://doi.org/10.1109/TIFS.2020.3021924>.
- Fei, W., Ohno, H., Sampalli, S., 2023. A systematic review of IoT security: research potential, challenges, and future directions. *ACM Comput. Surv.* 56 (5). <https://doi.org/10.1145/3625094>.
- Gaber, M.G., Ahmed, M., Janicke, H., 2023. Malware detection with artificial intelligence: a systematic literature review. *ACM Comput. Surv.* <https://doi.org/10.1145/3638552>.

Table 9Results of LSTM model on the *malware_api_class* data-set detailed for each class.

Metric	Adware	Backdoor	Downloader	Dropper	Spyware	Trojan	Virus	Worms
Precision	34.23%	31.50%	61.48%	37.44%	24.32%	21.52%	76.41%	42.42%
Recall	50.00%	40.00%	41.50%	41.01%	16.17%	24.00%	74.13%	42.00%
F1	40.64%	35.24%	49.55%	39.14%	19.42%	22.70%	75.25%	42.21%

Table 10Results of LSTM model on the *Alibaba Cloud Malware* data-set detailed for each class.

Metric	Normal	Ransom	miner	DDoS	Worm	Virus	Backdoor	Trojan
Precision	93.92%	87.23%	84.32%	53.51%	23.53%	93.73%	44.68%	51.92%
Recall	93.38%	87.23%	85.41%	58.58%	25.00%	90.77%	46.15%	53.79%
F1	93.65%	87.23%	84.86%	55.93%	24.24%	92.23%	45.41%	52.84%

**Fig. 10.** PCA projections on the *Alibaba Cloud Malware* data-set.**Table 11**Results of LSTM model on the *API Call Sequences* data-set detailed by each class.

Metric	Goodware	Malware
Precision	92.35%	99.59%
Recall	83.80%	99.82%
F1	87.86%	99.71%

Galloro, N., Polino, M., Carminati, M., Continella, A., Zanero, S., 2022. A systematical and longitudinal study of evasive behaviors in windows malware. *Comput. Secur.* 113, 102550. <https://doi.org/10.1016/j.cose.2021.102550>.

Gopinath, M., Sethuraman, S.C., 2023. A comprehensive survey on deep learning based malware detection techniques. *Comput. Sci. Rev.* 47, 100529. <https://doi.org/10.1016/j.cosrev.2022.100529>.

Guidotti, R., Monreale, A., Ruggieri, S., Pedreschi, D., Turini, F., Giannotti, F., 2018. Local rule-based explanations of black box decision systems. *arXiv:1805.10820* [abs].

Han, W., Xue, J., Wang, Y., Huang, L., Kong, Z., Mao, L., 2019. Maldae: detecting and explaining malware based on correlation and fusion of static and dynamic characteristics. *Comput. Secur.* 83, 208–233. <https://doi.org/10.1016/j.cose.2019.02.007>.

Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. *Neural Comput.* 9 (8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>.

Imran, M., Afzal, M.T., Qadir, M.A., 2015. Using hidden Markov model for dynamic malware analysis: first impressions. In: 2015 12th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD), pp. 816–821.

Jahromi, A.N., Hashemi, S., Dehghantanha, A., Parizi, R.M., Choo, K.K.R., 2020. An enhanced stacked lstm method with no random initialization for malware threat hunting in safety and time-critical systems. *IEEE Trans. Emerg. Top. Comput. Intell.* 4 (5), 630–640. <https://doi.org/10.1109/TETCI.2019.2910243>.

Jain, S., Wallace, B.C., 2019. Attention is not explanation. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Association for Computational Linguistics, Minneapolis, Minnesota, pp. 3543–3556. <https://www.aclweb.org/anthology/N19-1357>.

Jeon, S., Moon, J., 2020. Malware-detection method with a convolutional recurrent neural network using opcode sequences. *Inf. Sci.* 535, 1–15. <https://doi.org/10.1016/j.ins.2020.05.026>.

Jha, S., Prashar, D., Long, H.V., Taniar, D., 2020. Recurrent neural network for detecting malware. *Comput. Secur.* 99, 102037. <https://doi.org/10.1016/j.cose.2020.102037>.

Karn, R.R., Kudva, P., Huang, H., Suneja, S., Elfadel, I.M., 2021. Cryptomining detection in container clouds using system calls and explainable machine learning. *IEEE Trans. Parallel Distrib. Syst.* 32 (3), 674–691. <https://doi.org/10.1109/TPDS.2020.3029088>.

Kiefer, S., 2022. CaSE: explaining text classifications by fusion of local surrogate explanation models with contextual and semantic knowledge. *Inf. Fusion* 77, 184–195. <https://doi.org/10.1016/j.inffus.2021.07.014>.

Kokalj, E., Škrlić, B., Lavrač, N., Pollak, S., Robnik-Šikonja, M., 2021. BERT meets Shapley: extending SHAP explanations to transformer-based classifiers. In: Proceedings of the EACL Hackshop on News Media Content Analysis and Automated Report Generation. Online: Association for Computational Linguistics, pp. 16–21. <https://www.aclweb.org/anthology/2021.hackashop-1.3>.

Li, C., Lv, Q., Li, N., Wang, Y., Sun, D., Qiao, Y., 2022a. A novel deep framework for dynamic malware detection based on API sequence intrinsic features. *Comput. Secur.* 116, 102686. <https://doi.org/10.1016/j.cose.2022.102686>.

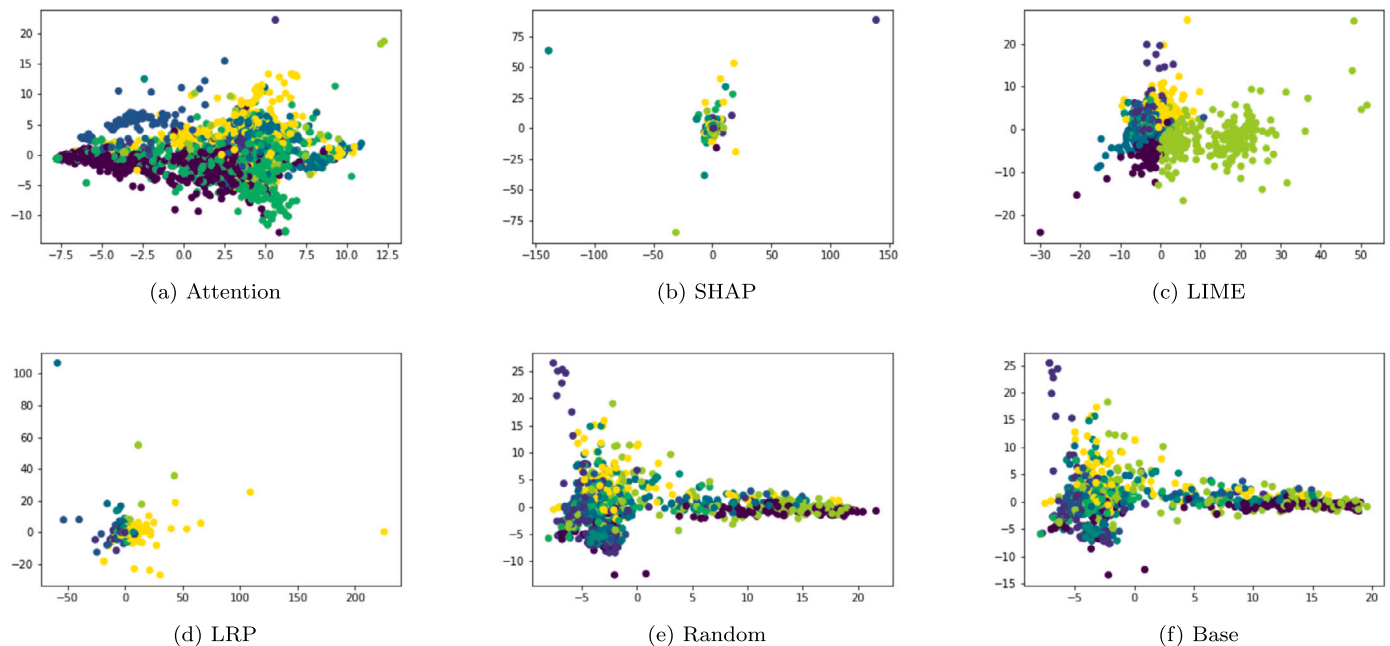


Fig. 11. PCA projections on the *malware_api.class* data-set detailed for each class.

- Li, Q., Peng, H., Li, J., Xia, C., Yang, R., Sun, L., Yu, P.S., He, L., 2022b. A survey on text classification: from traditional to deep learning. *ACM Trans. Intell. Syst. Technol.* 13 (2). <https://doi.org/10.1145/3495162>.
- Liu, Y., Tantithamthavorn, C., Li, L., Liu, Y., 2022a. Deep learning for Android malware defenses: a systematic literature review. *ACM Comput. Surv.* 55 (8), 1–36. <https://doi.org/10.1145/3544968>.
- Liu, Y., Tantithamthavorn, C., Li, L., Liu, Y., 2022b. Explainable ai for Android malware detection: towards understanding why the models perform so well? In: 2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE), pp. 169–180.
- Lundberg, S.M., Lee, S.I., 2017. A unified approach to interpreting model predictions. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf.
- Marques, P., Rhode, M., Gashi, I., 2021. Waste not: using diverse neural networks from hyperparameter search for improved malware detection. *Comput. Secur.* 108, 102339. <https://doi.org/10.1016/j.cose.2021.102339>.
- Melis, M., Maiorca, D., Biggio, B., Giacinto, G., Roli, F., 2018. Explaining black-box Android malware detection. In: 2018 26th European Signal Processing Conference (EUSIPCO). IEEE, pp. 524–528.
- Mills, R., Marnerides, A.K., Broadbent, M., Race, N., 2022. Practical intrusion detection of emerging threats. *IEEE Trans. Netw. Serv. Manag.* 19 (1), 582–600. <https://doi.org/10.1109/TNSM.2021.3091517>.
- Nguyen, D., 2018. Comparing automatic and human evaluation of local explanations for text classification. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 1069–1078.
- Oksuz, K., Cam, B.C., Kalkan, S., Akbas, E., 2022. One metric to measure them all: localisation recall precision (LRP) for evaluating visual detection tasks. *IEEE Trans. Pattern Anal. Mach. Intell.* 44 (12), 9446–9463. <https://doi.org/10.1109/TPAMI.2021.3130188>.
- de Oliveira, A.S., Sassi, R.J., 2023. Behavioral malware detection using deep graph convolutional neural networks. *Authorea Preprints*.
- Pei, X., Yu, L., Tian, S., 2020. Amalnet: a deep learning framework based on graph convolutional networks for malware detection. *Comput. Secur.* 93, 101792. <https://doi.org/10.1016/j.cose.2020.101792>.
- Pendleton, M., Garcia-Lebron, R., Cho, J.H., Xu, S., 2016. A survey on systems security metrics. *ACM Comput. Surv.* 49 (4). <https://doi.org/10.1145/3005714>.
- Qiu, L., Yang, Y., Cao, C.C., Zheng, Y., Ngai, H., Hsiao, J., Chen, L., 2022. Generating perturbation-based explanations with robustness to out-of-distribution data. In: *WWW '22 Proceedings of the ACM Web Conference 2022*. Association for Computing Machinery, New York, NY, USA, pp. 3594–3605.
- Ribeiro, M.T., Singh, S., Guestrin, C., 2016. “Why should I trust you?": explaining the predictions of any classifier. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- Rong, Y., Leemann, T., Nguyen, T.T., Fiedler, L., Qian, P., Unhelkar, V., Seidel, T., Kasneci, G., Kasneci, E., 2023. Towards human-centered explainable AI: a survey of user studies for model explanations. *IEEE Trans. Pattern Anal. Mach. Intell.*, 1–20. <https://doi.org/10.1109/TPAMI.2023.3331846>.
- Saeed, W., Omlin, C., 2023. Explainable AI (XAI): a systematic meta-survey of current challenges and future opportunities. *Knowl.-Based Syst.* 263, 110273. <https://doi.org/10.1016/j.knosys.2023.110273>.
- Samek, W., Binder, A., Montavon, G., Lapuschkin, S., Müller, K.R., 2016. Evaluating the visualization of what a deep neural network has learned. *IEEE Trans. Neural Netw. Learn. Syst.* 28 (11), 2660–2673. <https://doi.org/10.1109/TNNLS.2016.2599820>.
- Santos, I., Peña, Y.K., Devesa, J., Bringas, P.G., 2009. N-grams-based file signatures for malware detection. *ICEIS 2* (9), 317–320. <https://doi.org/10.5220/0001863603170320>.
- Schlegel, U., Keim, D.A., 2023. A deep dive into perturbations as evaluation technique for time series XAI. In: *World Conference on Explainable Artificial Intelligence*. Springer, pp. 165–180.
- Shapley, L.S., et al., 1988. *A Value for N-Person Games*. Princeton University Press, Princeton.
- Sun, N., Ding, M., Jiang, J., Xu, W., Mo, X., Tai, Y., Zhang, J., 2023. Cyber threat intelligence mining for proactive cybersecurity defense: a survey and new perspectives. *IEEE Commun. Surv. Tutor.* 25 (3), 1748–1774. <https://doi.org/10.1109/COMST.2023.3273282>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need. *CoRR*. arXiv:1706.03762.
- Warnecke, A., Arp, D., Wressnegger, C., Rieck, K., 2020. Evaluating explanation methods for deep learning in security. In: 2020 IEEE European Symposium on Security and Privacy (EuroS&P), pp. 158–174.
- Wiegrefe, S., Pinter, Y., 2019. Attention is not not explanation. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, Hong Kong, China, pp. 11–20.
- Wu, B., Chen, S., Gao, C., Fan, L., Liu, Y., Wen, W., Lyu, M.R., 2021. Why an Android app is classified as malware: toward malware classification interpretation. *ACM Trans. Softw. Eng. Methodol.* 30 (2). <https://doi.org/10.1145/3423096>.
- Yadav, P., Menon, N., Ravi, V., Vishwanathan, S., Pham, T.D., 2022. EfficientNet convolutional neural networks-based Android malware detection. *Comput. Secur.* 115, 102622. <https://doi.org/10.1016/j.cose.2022.102622>.
- Yan, S., Ren, J., Wang, W., Sun, L., Zhang, W., Yu, Q., 2023. A survey of adversarial attack and defense methods for malware classification in cyber security. *IEEE Commun. Surv. Tutor.* 25 (1), 467–496. <https://doi.org/10.1109/COMST.2022.3225137>.
- Yang, Z., Yang, D., Dyer, C., He, X., Smola, A., Hovy, E., 2016. Hierarchical attention networks for document classification. In: *HLT-NAACL*, pp. 1480–1489.
- Zhang, W.E., Sheng, Q.Z., Alhazmi, A., Li, C., 2020. Adversarial attacks on deep-learning models in natural language processing: a survey. *ACM Trans. Intell. Syst. Technol.* 11 (3). <https://doi.org/10.1145/3374217>.
- Zhang, Y., Song, K., Sun, Y., Tan, S., Udell, M., 2019. “Why should you trust my explanation?” Understanding uncertainty in lime explanations. *arXiv: Learning*.
- Zhang, Z., Hamadi, H.A., Damiani, E., Yeun, C.Y., Taher, F., 2022. Explainable artificial intelligence applications in cyber security: state-of-the-art in research. *IEEE Access* 10, 93104–93139. <https://doi.org/10.1109/access.2022.3204051>.

Zhong, F., Cheng, X., Yu, D., Gong, B., Song, S., Yu, J., 2023. MalFox: camouflaged adversarial malware example generation based on conv-GANs against black-box detectors. *IEEE Trans. Comput.*, 1–14. <https://doi.org/10.1109/TC.2023.3236901>.



Antonio Galli is a Research Fellow at the Department of Electrical Engineering and Information Technologies of University of Naples Federico II. He holds a Bachelor's Degree (2016) and Master's Degree cum laude (2019) in Computer Science and Engineering from the University of Naples "Federico II". His main research interests are in the area of Deep Learning and Big Data analytics.



Valerio La Gatta is a PhD student in Information Technology and Electrical Engineering at the Department of Electrical Engineering and Information technology of the University of Naples Federico II. He received the Master degree in Computer engineering from the University of Naples Federico II in 2020. His research interests are focused on Social Network Analysis, eXplainable Artificial Intelligence, Graph Data Mining.



Vincenzo Moscato is currently an Associate Professor of Database and Information Systems at Department of Electrical Engineering and Information Technologies of University of Naples Federico II, where he received the Ph.D in Computer Science and Engineering. His current research interests lie in the area of multimedia, knowledge management and Big Data analytics. He was involved in several international, national and local research projects and at present is an author of more than one hundred publications on international journal and conference proceedings.



Marco Postiglione is a PhD student in Information and Communication Technology for Health (ICTH) at the Department of Electrical and Computer Engineering of the University of Naples Federico II. He received the Master's degree in Computer Engineering from the University of Naples Federico II in 2020. His research interests are focused on eXplainable Artificial Intelligence, Graph Data Mining and Natural Language Processing.



Giancarlo Sperli is an assistant professor at the Department of Electrical Engineering and Information Technology of the University of Naples Federico II. He obtained his PhD in Information Technology and Electrical Engineering at the same University defending his thesis: "Multimedia Social Networks". He is a member of the Pattern analysis and Intelligent Computation for multimedia Systems (PICUS) departmental research groups. His main research interests are in the area of Cybersecurity, Multimedia data analysis and Social Networks Analysis. He served as guest editor of different special issues on International Journals.

Finally, he has authored about 108 publications in international journals, conference proceedings and book chapters.